

Jeeves — Day Planner Usability & Abilities PRD

Status: proposal · **Date:** 04 Aug 2026 · **Supersedes:** the planner portions of `UI-AUDIT.md` (2026-07-31), which this document reconciles against the *current* code.

Reconciliation note. The 31-Jul audit is stale against the tree it audited. Two of its three P0 items are **already shipped**: - The Doherty-threshold (9-min silent spinner) is fixed — `PlanningProgressCard` (elapsed counter + expectation copy + *Stop and keep the current plan*) is wired into `DayPlannerView (.planBar, ~line 438)`. - Dynamic Type is largely in — `Font.ui(_:weight:design:)` (UIFontMetrics-backed) has replaced the fixed `.system(size:)` call sites the audit counted. This PRD therefore targets the gaps the audit *didn't* see because they were built after it, plus the friendliness issues that remain inside the planner specifically. Every claim is grounded in a file/line you can open.

1. Purpose & scope

Make the **Day Planner tab** the place you both *read* and *shape* your day — not just the surface that triggers a chat-style generation and then hands control back to Claude. Today the planner is strong at *display* and *generation*, but weak at *direct manipulation*: a user who wants to nudge a block, add a one-off, understand a drop, or recover a delete must either open the editor (limited) or leave for chat.

In scope: Planner tab UX, the plan timeline, the plan editor, planner-level interactions, and planner-only abilities.

Out of scope: the chat engine, travel mode internals (covered by its own guards/PRD §5.5), CloudKit sync, the auto-planner backend timing.

2. What's already solid (don't re-litigate)

- **Honest waiting state.** `PlanningProgressCard` narrates stages, ticks elapsed time, sets a 4-min expectation, and offers cancel with a real outcome ("keep the current plan"). This is the best UX in the app — keep its posture.
- **Travel stands down cleanly.** `travelSuggestionBanner` + `TravelQuietDayCard` handle travel days without the routine plan. Good.
- **Stale-plan banner.** `stalePlanBanner` tells the user the plan is out of date and offers a rebuild. Exists, good.
- **Adherence loop.** `planCard` shows a live % followed, lets you toggle done/skipped, and starts logged sessions. Mature.
- **Activity picker.** `ActivityPickerSheet` lets you choose the day's routine activities before planning — the "not today" answer lives beside the plan button.

The friendliness problem is **not** the core loop; it's everything *around* it.

3. Usability gaps (grounded)

#	Gap	Evidence	Why it bites
U1	Deleted things in the planner are unrecoverable. <code>UndoBanner</code> + <code>UndoableDelete</code> are fully built (<code>UndoBanner.swift</code>) but referenced 0 times in the app. Event/trip deletes in the planner have no undo.	<code>UndoBanner.swift</code> exists; grep for <code>UndoBanner/UndoableDelete</code> = 0 hits.	A mis-tap on a trip row loses hand-entered travel data with no way back. Nielsen #3.

#	Gap	Evidence	Why it bites
U2	You can't add a one-off block from the planner. Only the <i>routine</i> activity picker exists (<code>ActivityPickerSheet</code> → <code>planMyDay</code>). A "call at 3, 30 min" must be typed into chat.	<code>planBar</code> checklist button → <code>pickerSheet</code> . No custom free-text insert path in <code>DayPlannerView/PlanEditorView</code> .	The most common real-world edit ("slot this in") has no UI; it's chat-only.
U3	The editor can't remove or insert a block. <code>PlanEditorView</code> only retimes/edits/reorders <i>existing</i> blocks (<code>onMove</code> + <code>BlockDetailEditor</code>). No delete, no insert.	<code>PlanEditorView.swift</code> body: <code>ForEach(blocks) + .onMove</code> only.	To drop something the planner forced in, you either live with it or go to chat.
U4	Drop/shrink reasons aren't surfaced inline. <code>GeneratedPlan</code> carries <code>dropped/shrunk</code> , but the post-plan timeline shows no "what I left out and why" receipt at the point of decision.	Audit U3; no "dropped"/"why" text in <code>DayPlannerView</code> plan surface.	The user learns <i>after</i> the fact, in chat, why prep got cut — violating the app's own "receipts, not optimism" principle <i>in the UI</i> .
U5	No live position on the timeline. The day's blocks render statically; there's no "you are here" marker and no dimming of elapsed blocks.	<code>PlanTimelineCard</code> (not read in full; no now-marker in <code>DayPlannerView</code>).	On a phone you glance at, "what's next / what's done" should be one look, not scrolling.
U6	Date dial doesn't reveal trip days or planned days. Known limitation, PRD §12: "Date dial doesn't mark trip spans."	PRD.md §12.	Scrolling the dial, you can't see which days are travel vs planned vs empty — navigation is blind.
U7	Re-plan hides a real behavior. Known defect, PRD §12: "replan treats elapsed as done." The <i>Re-plan</i> button (<code>planBar</code>) offers no acknowledgement that past blocks are locked as completed and only the remainder rebuilds.	PRD.md §12; <code>planMyDay</code> path.	A user re-planning because they're <i>late</i> gets a plan that assumes the late block finished — the exact wrong case.
U8	Sparse empty states. No plan, no events → a near-empty screen. <code>ContentUnavailableView</code> is used 0 times app-wide.	grep <code>ContentUnavailableView = 0</code> .	First-run / quiet-day users get bare text, lowest-contrast treatment.
U9	Icon-only controls lack labels. <code>accessibilityLabel</code> appears in only ~6 places; the planner's edit/checklist/date-jump icons are unlabeled.	grep <code>accessibilityLabel</code> ≈ 6 hits app-wide.	VoiceOver announces "Button"; the audit's accessibility score (5/20) still applies to the planner.

4. Proposed enhancements

A — Usability fixes (close the friendliness gap)

- **A1 · Inline drop/shrink receipt.** After a plan (or re-plan), render a compact "Left out / shortened" section in `planCard` listing each dropped/shrunk item and its reason, sourced from `plan.dropped/plan.shrunk`. Tapping expands the reason. *Honors the app's "receipts, not optimism" rule in the UI itself.*

- **A2 · Wire UndoBanner into planner deletes.** Attach a `@State private var pendingDelete: UndoableDelete?` to `DayPlannerView`; on event/trip/reminder delete, present the banner and defer the actual `modelContext.delete` until the timeout or perform it immediately + offer restore. *(Banner already built; this is integration, not new code.)*
- **A3 · Add a one-off block from the planner.** A "+" affordance on the timeline / in the editor that opens a lightweight sheet: title, kind, start time, duration. Inserts into the committed plan via `PlanEditLogic.retime`. No Claude needed.
- **A4 · Delete / skip a block in the editor.** Extend `PlanEditorView` with swipe-to-delete on non-anchor rows (anchors stay locked) and a "skip, don't delete" option. Re-times around the gap.
- **A5 · Pin a block.** Long-press or a pin control marks a block `frozen`; re-plan and the editor preserve its time. Stored as a flag on `DailyPlanState` per block key. *Prevents the "why did my standing 7am block move" frustration.*
- **A6 · Live "now" marker + elapsed dimming.** A horizontal rule labeled with the current time on `PlanTimelineCard`; blocks before now render muted. Pure view logic, no model change.
- **A7 · Enriched date dial.** Mark each dial day with a small glyph: 🚗 travel-covered, ● has a committed plan, ■ empty. Reads `trips` + `DailyPlanState` per visible day. *(PRD §12 known limitation, now buildable.)*
- **A8 · Empty-state cards.** `ContentUnavailableView` for "No plan yet" (with Plan-my-day CTA) and "No events" (with calendar-import CTA). Native component, one line each.
- **A9 · Re-plan honesty.** When re-planning a *today* past its start, show a one-line note: "Blocks already underway are kept as done; I'll rebuild the rest." Surfaces the PRD §12 behavior instead of hiding it.
- **A10 · Finish accessibility on planner controls.** Add `accessibilityLabel/accessibilityHint` to the edit, checklist, date-jump, and travel buttons (the `AppNavigation.swift` pattern). Dynamic Type is mostly done; this closes the remaining 14 view files.

B — New abilities (features)

- **B1 · Plan my week.** A week strip above the day view showing each day's plan-density / travel / conflicts; tap a day to jump. The auto-planner already fills a rolling window — surface it. *(High value, natural extension of A7.)*
- **B2 · Reuse a past day's plan.** "Use last Tuesday's plan" — clone a prior `DailyPlanState.plan` into today via `PlanEditLogic`. Cheap, frequently wanted.
- **B3 · Direct timeline drag.** Drag a block to a new time on `PlanTimelineCard`; non-anchors re-time live, anchors snap back with a hint. More intuitive than the editor's reorder handles.
- **B4 · Conflict surfacing.** When a calendar event overlaps a planned block, tint the overlap and show "Event X clashes with prep — re-plan or move." Turns silent collisions into visible ones.
- **B5 · Today widget / Lock Screen.** A `WidgetKit` extension showing the next 1–2 blocks + leave-by. Notifications exist; a glanceable widget is the missing always-on surface. *(New target/extension; additive.)*
- **B6 · Share a day.** Export the day's plan as text/ image ("send to a partner"). Read-only, no new data model.
- **B7 · Draft vs committed.** Plan a future day that already has a plan into a *draft*; commit only on confirm. Protects the existing plan from an exploratory re-plan.
- **B8 · Per-day plan history.** See how Tuesday's plan changed across generations (`PlanGenerationLog` already records attempts). "What did Jeeves change?" view.
- **B9 · Natural-language add in-planner.** A small inline field "Add anything..." on the planner that routes to the same chat tool `add_block` uses, so a one-off doesn't require opening the chat tab. *(Thin wrapper over existing tool.)*
- **B10 · Focus-mode tie-in.** When a deep-work block is live, offer to enable iOS Focus (Do Not Disturb) for its duration. System integration, high delight.

5. Prioritization (value per unit of work)

Rank	Item	Effort	Value	Notes
1	A1 drop/shrink receipt	S	High	Honors core principle; pure view + existing data
2	A2 wire UndoBanner	S	High	Code exists; integration only; real data-loss fix
3	A3 add one-off block	M	High	Most-requested real edit has no UI
4	A6 now-marker + dimming	S	Med-High	One glance tells you where you are
5	A7 trip/plan glyphs on dial	S	Med	PRD §12 known gap, cheap
6	A9 re-plan honesty	XS	Med	One line, kills a silent defect
7	A5 pin a block	M	Med	Prevents recurring frustration
8	A4 delete/skip in editor	M	Med	Completes the editor
9	A8 empty states	S	Med	First-run polish
10	A10 a11y labels	M	Med	Audit 5/20 still applies
11	B2 reuse past day	S	Med	Cheap, loved
12	B1 plan my week	L	High	Biggest new ability; depends on A7
13	B3 timeline drag	L	High	Best manipulation UX; most work
14	B4 conflict surfacing	M	Med	Visible collisions
15	B5 widget	L	Med	New extension; additive
16	B6/B9 share / inline NL	M	Med	Thin wrappers
17	B7/B8/B10 draft/history/focus	M-L	Low-Med	Nice, lower urgency

Recommended first build (a coherent "planner you can shape" milestone): A1 → A2 → A3 → A6 → A7 → A9. Together they make the planner trustworthy (you see drops, you can undo, you can add, you know where you are, you can navigate, you understand re-plan) with only small/medium effort and **no new targets or data models**.

6. Decisions needed from you

1. **Scope of first pass** — just the A-series usability fixes, or include a headline ability (B1 week / B3 drag)? 2. **Re-plan semantics (U7/A9)** — confirm we should *surface* "elapsed kept as done" rather than change the behavior (changing it is a bigger engine change; surfacing is free and honest). 3. **Undo model (A2)** — defer-delete (item vanishes, restore within 6s) vs soft-delete-then-purge. The built **UndoBanner** assumes defer-delete; confirm that matches your mental model. 4. **Widget (B5)** — acceptable to add a **WidgetKit** extension target to the project? It's the one item that touches the build's target list.

7. Non-goals

- Changing the Claude-first generation model or the auto-planner backend timing (separate track; PRD §12 observation window to ~2026-08-28).
- Touching travel-mode internals (guarded, PRD §5.5).

- Multi-user / accounts / server (explicit non-goal, PRD §12).
 - Dark mode (product call, not a planner defect).
-

8. Verification (how we'll know it shipped)

Every item keeps the app's QA doctrine:

- **Pure helpers** (drop-reason formatting, dial glyphs, now-marker math, re-plan note logic) get **unit tests** alongside — the app's 404-test gate stays green.
- **View changes** get a device check (the audit's own gap: it never ran a screen).
- **Undo / add / delete** paths get added to **ChatToolParityTests**-style or new **DayPlannerTests** covering the store end-state, not just the UI.
- **No secret, no new network call** without Keychain; widget is local-only.